

Bellman-Ford algoritam i njegova primjena za određivanje optimalne staze

1st Berina Biberović

Odsjek za automatiku i elektroniku

Elektrotehnički fakultet

Univerzitet u Sarajevu

Sarajevo, Bosna i Hercegovina

bbiberovic1@etf.unsa.ba

2nd Ahmed Imamović

Odsjek za automatiku i elektroniku

Elektrotehnički fakultet

Univerzitet u Sarajevu

Sarajevo, Bosna i Hercegovina

aimamovic6@etf.unsa.ba

Sažetak—U brojnim primjenama teorije grafova potrebno je naći najkraći put (optimalnu stazu) koji povezuje dva zadana čvora. Ovaj rad pruža pregled Bellman-Ford algoritma koji upravo rješava navedeni problem pronalaska najkraćeg puta. U prvom dijelu rada je opisan problem najkraćeg puta i predstavljen je Bellman-Ford algoritam. Objašnjena je osnovna ideja i koraci izvođenja, te je urađena analiza kompleksnosti Bellman-Ford algoritma. Drugi dio rada analizira Bellman-Ford algoritam u poređenju sa Dijkstrinim i Floyd-Warshallovim algoritmom, gdje su istaknute prednosti i ograničenja Bellman-Ford algoritma u odnosu na ova dva navedena algoritma. Kroz zaključak ističemo važnost Bellman-Ford algoritma i njegovu primjenu u različitim područjima.

Glavni pojmovi—Bellman-Ford algoritam, najkraći put

Abstract—A common problem found in applications of graph theory is finding the shortest path (i.e. the optimal path) that connects two vertices. This paper presents the Bellman-Ford algorithm as the solution to the optimal path problem. The first section of this paper describes the shortest path problem and describes the Bellman-Ford algorithm. The basic idea of this algorithm and the runtime steps are explained, then a complexity analysis is done. In the second section of this paper the Bellman-Ford algorithm is compared to the Dijkstra and Floyd-Warshall algorithms, where the advantages and drawbacks of the Bellman-Ford algorithm are compared to these other two algorithms. Through the conclusion, the importance of the Bellman-Ford algorithm and its usage in different fields are highlighted.

Keywords—Bellman-Ford algorithm, optimal path

I. UVOD

U brojnim primjenama teorije grafova potrebno je naći najkraći put koji povezuje dva zadana čvora x_i i x_j . U slučaju razmatranja težinskog grafa pod dužinom puta se smatra suma težina svih grana koje čine put. Za ovaj problem predloženi su brojni efikasni algoritmi. Jedan od najstarijih takvih algoritama je Dantzingov algoritam, a danas se koriste njegove poboljšane verzije kao što su Dijkstrin algoritam i Floyd-Warshallov algoritam. Prethodno pomenuti problem se može proširiti tako da grafovi sadrže i negativne težine grana. Negativne težine grana mogu djelovati besmisleno, međutim u praksi se mogu koristiti za opisivanje pojava poput finansijskih transakcija (dobitak/gubitak), oslobađanje ili apsorpcija toplote u hemijskim reakcijama itd. Negativne težine grana javljaju se i u slučajevima kada se neki problemi, koji nemaju direktne veze s problemom nalaženja najkraćeg puta, transformiraju u

ekvivalentan problem nalaženja najkraćeg puta[1]. Međutim, Dijkstrin algoritam ne radi ispravno za grafove u kojima postoje grane s negativnim težinama, a Floyd-Warshallov algoritam, u svom osnovnom obliku, ne može detektovati pojavu negativnog ciklusa u grafu. Stoga je za takve grafove potrebno koristiti neke druge algoritme, kao što je upravo Bellman-Fordov algoritam.

Mada se Bellman-Fordov algoritam može koristiti i s grafovima kod kojeg sve grane imaju pozitivne težine, on je najčešće primjetno neefikasniji od Dijkstrinog algoritma. Bellman-Fordov algoritam je nastao unapređenjem tzv. Bellmanovog algoritma za nalaženje najkraćeg puta u usmjerenim acikličkim grafovima, tj. Bellman-Fordov algoritam je uopćenje Bellmanovog algoritma na proizvoljne grafove. Postoje dvije verzije ovog algoritma: Bellmanova i Fordova. Bellmanova verzija neposrednije prati ideje izvornog Bellmanovog algoritma, ali je nešto nezgrapnija za rad (bilo ručno ili mašinski)[1]. Stoga se u praksi više koristi Fordova verzija, koja će u nastavku biti opisana.

II. KORIŠTENI ALGORITAM

A. Terminologija i notacija

Prije objašnjavanja algoritma predstaviti će se osnovna terminologija i notacija korištena u matematskom opisu algoritma. Težinski graf se definira kao uređena trojka $D = (V(D), A(D), c)$, pri čemu su elementi skupa V čvorovi grafa, elementi skupa A grane/lukovi grafa, dok je $c : A \rightarrow \mathbb{R}$ funkcija koja svakoj grani iz A pridružuje neki realan broj, odnosno njenu težinu[1]. U orijentiranom grafu za granu koja spaja čvorove u i v , tako da je usmjerena od u ka v , kaže se da izlazi iz čvora u , a ulazi u čvor v te se označava se kao (u, v) ili kraće uv . Dva čvora su susjedna ako su spojena granom. Skup ulaznih susjeda čvora v predstavlja sve susjedne čvorove tako da je grana usmjerena od susjednog čvora ka čvoru v , tj. $N_D^-(v) = \{w \in V \setminus v : vw \in A\}$. Za čvorove x i y u grafu D (u kome nema negativnih ciklusa), definiše se *udaljenost* od x do y i označava se $dist(x, y)$. Udaljenost predstavlja minimalnu dužinu (x, y) -šetnje (engl. *walk*), ako su čvorovi povezani, inače je $dist(x, y) = \infty$. Specijalan slučaj predstavlja $dist(x, x) = 0$ [2]. Usmjereni graf je jako povezan

ako za svaka dva čvora x_i i x_j postoji kako put iz čvora x_i u x_j tako i put iz čvora x_j u čvor x_i [1].

B. Matematski dokaz Bellman-Ford algoritma

Neka je $D = (V, A, c)$ težinski orijentirani graf tako da neke grane mogu imati negativnu težinu. Primjenom algoritma objašnjenog u nastavku mogu se pronaći rastojanja od datog čvora s u D do svih ostalih čvorova, uz uslov da D ne sadrži negativan ciklus.

Neka je $\delta(v, m)$ dužina najkraćeg (s, v) -puta sa najviše do m grana. Očigledno vrijedi, $\delta(s, 0) = 0$ i $\delta(v, 0) = \infty$ za svaki $v \in V \setminus s$. Neka je $v \in V$. Dokažimo da za svako $m \geq 0$,

$$\delta(v, m+1) = \min\{\delta(v, m), \min\{\delta(u, m) + c(u, v)\}\} \quad (1)$$

gdje je $u \in N_D^-(v)$. Dokažimo jednakost 1 indukcijom po m . Za $m = 0$, jednakost je trivijalna. Za dalje potrebe se navodi slijedeća tvrdnja[2]:

Tvrdnja 2.1: Ako je $P = x_1 \dots x_k$ najkraći (x_1, x_k) -put u D , onda je $P[x_i, x_j]$ najkraći (x_i, x_j) -put za sve $1 \leq i \leq j \leq k$.

Sada se za $m \geq 1$ prezentuje slijedeći argument. Neka postoji najkraći (s, v) -put P sa ne više od $m+1$ grana. Ako P sadrži najviše m grana, njegova dužina je $\delta(v, m)$. Inače, P sadrži $m+1$ granu te se uz tvrdnju 2.1 zaključuje da sadrži najkraći (s, u) -put sa m grana i granu uv za neko $u \in N_D^-(v)$. Ako svaki najkraći (s, v) -put ima više od $m+1$ grana, onda ne postoji ulazni susjedni čvor u od čvora v takav da $\delta(u, m) < \infty$. Dakle, jednakost 1 korektno implicira da je $\delta(v, m+1) = \infty$.

Kako nijedan put nema više od $n-1$ grana, $\delta(v, n-1) = \text{dist}(s, v)$ za svaki $v \in V \setminus s$. Tako koristeći jednakost 1 za $m = 0, 1, \dots, n-2$, dobijamo udaljenosti od s do svih čvorova iz D . Ovo rezultira slijedećim algoritmom[2].

C. Bellman-Ford algoritam

Ulaz: Težinski usmjereni graf $D = (V, A, c)$ bez negativnih ciklusa i fiksiranim čvorom $s \in V$.

Izlaz: Parametar δ_v za svaki $v \in V$ takav da $\delta_v = \text{dist}(s, v)$ za sve $v \in V$.

1) Postaviti $\delta_s := 0$ i $\delta_v := \infty$ za svako $v \in V \setminus s$.

2) Za $i = 1$ do $n-1$ uradi: za svako $vu \in A$ ažurirati parametar δ_u postavljajući $\delta_u = \min\{\delta_u, \delta_v + c(v, u)\}$.

Lahko se pokaže da je kompleksnost ovog algoritma $\mathcal{O}(nm)$, pa slijedi teorem[2].

Teorem 2.1: Kada se primijeni da težinski usmjereni graf $D = (V, A, c)$ bez negativnih ciklusa i fiksiranim čvorom $s \in V$, Bellman-Fordov algoritam ispravno odredi udaljenosti od s do svih ostalih čvorova u D u vremenu $\mathcal{O}(nm)$.

Za ispitivanje postojanosti negativnog ciklusa pretpostavi se da je graf D jako povezan, te se dodaje treći korak u prethodno navedeni algoritam:

3) Za svaku granu $vu \in A$ uradi : ako vrijedi $\delta_u > \delta_v + c(vu)$ vrati poruku 'graf sadrži negativan ciklus'

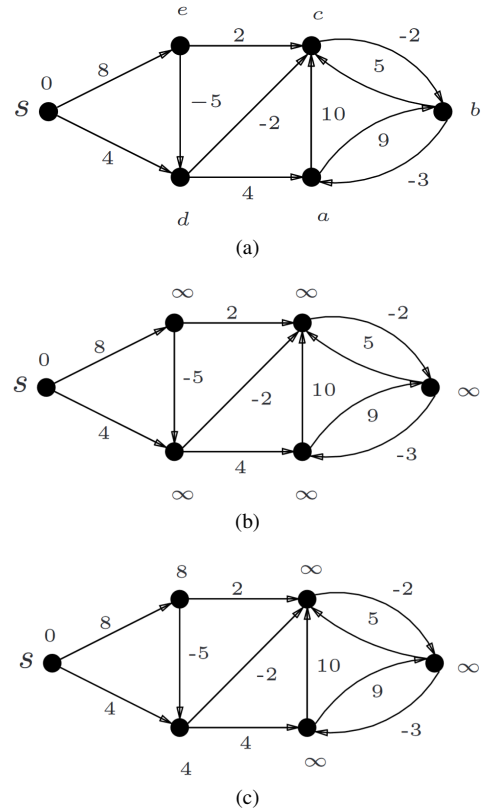
Teorem 2.2: Jako povezani težinski usmjereni graf D sadrži negativan ciklus ako i samo ako korak 3 vrati svoju poruku.

Dokaz: Neka D ne sadrži negativan ciklus. Uz korak 2 algoritma i tvrdnju 2.1, vrijedi $\delta_u \leq \delta_v + c(vu)$ za svaku granu $vu \in A$. Dakle, poruka se neće vratiti.

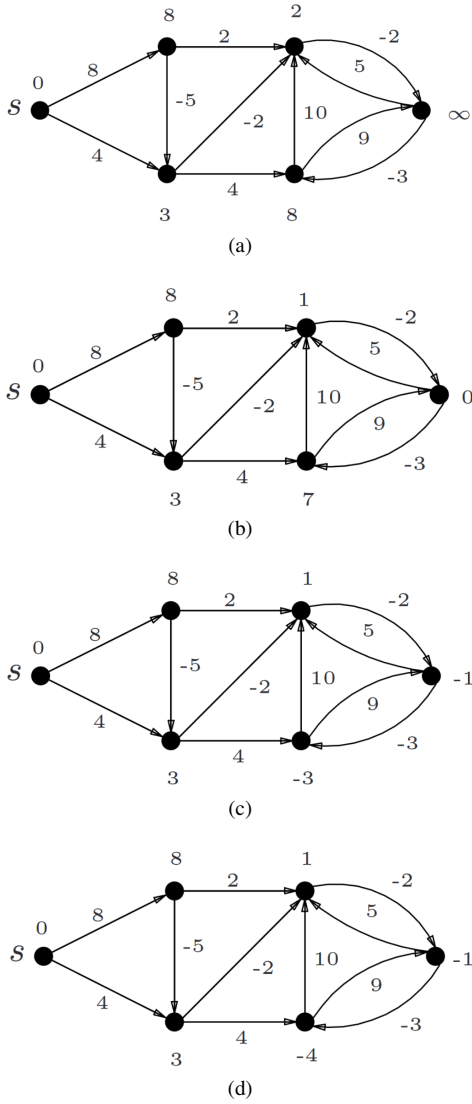
Neka je sada pretpostavka da D sadrži negativan ciklus $Z = v_1 v_2 \dots v_k v_1$. U svrhu kontradikcije, pretpostaviti da korak 3 Bellman-Fordovog algoritma nije vratio odgovarajuću poruku. Tako vrijedi $\delta_{v_i} \leq \delta_{v_{i-1}} + c(v_{i-1} v_i)$ za svako $i \in [k]$, gdje je $v_0 = v_k$. Dakle vrijedi,

$$\sum_{i=1}^k \delta_{v_i} \leq \sum_{i=1}^k \delta_{v_{i-1}} + \sum_{i=1}^k c(v_{i-1} v_i)$$

Kako su prve dvije sume u posljednjoj nejednakosti jednake, dobije se $0 \leq \sum_{i=1}^k c(v_{i-1} v_i) = c(Z)$, što je kontradikcija[2]. Slike 1 i 2 prikazuju izvršavanje Bellman-Fordovog algoritma.



Slika 1. Izvršavanje Bellman-Ford algoritma. Za unutarnju petlju drugog koraka algoritma grane su razmatrane po leksikografskom slijedu: $ab, ac, ba, bc, cb, da, dc, ec, ed, sd, se$. a) Ulazni graf sa označenim čvorovima i težinom grana. b) Zadate su početne vrijednosti parametra δ uz čvorove. Stanje nakon prvog koraka algoritma c) Stanje nakon prve iteracije drugog koraka algoritma[2].



Slika 2. (a) do (d) Stanja nakon svake uzastopne iteracije unutarnje petlje drugog koraka algoritma do kraja[2].

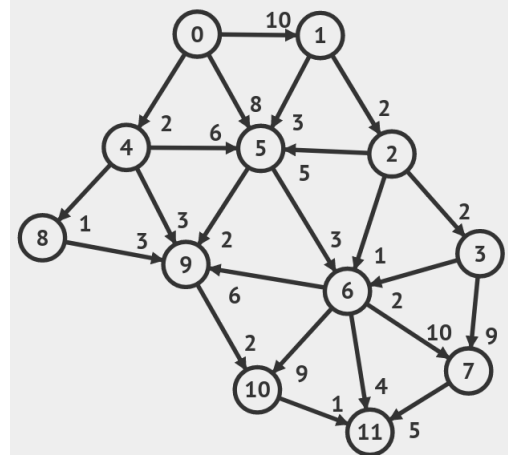
III. SIMULACIJSKI REZULTAT

Potpuna implementacija prethodno objašnjenog Bellman-Ford algoritma je data u prilogu ovog rada. U prilogu su isto tako date implementacije Dijkstrinog i Floyd-Warshallovog algoritma. U nastavku će biti objašnjeni glavni dijelovi sva tri algoritma kao i njihova usporedba.

Komputacijska kompleksnost algoritma predstavlja količinu vremena, memorije ili drugih resursa kako bi se algoritam izvršio. U ovom radu je korišten broj iteracija prilikom računanja najkraćeg puta kao parametar za računanje kompleksnosti navedena tri algoritma.

Za Bellman-Ford algoritam, komputacijska kompleksnost je $\mathcal{O}(n \cdot m)$ (u najgorem slučaju), dok za Dijkstrin algoritam je $\mathcal{O}(m + n \cdot \log n)$ (u najgorem slučaju) i za Floyd-Warshallov algoritam je $\mathcal{O}(n^3)$ (ista je kompleksnost i za najbolji i za najgori slučaj). U prethodnim formulama n predstavlja broj čvorova a m predstavlja broj grana unutar grafa.

Neka je dat graf kao na slici 3 (postavka grafa je preuzeta sa [3], dok je slika grafa je preuzeta sa [4]). Potrebno je pronaći najkraći put od 0-tog do 11-tog čvora.



Slika 3. Testni graf

Svi algoritmi koriste .txt datoteku u kojoj je opisan graf kako bi generisali traženi minimalni put od početnog do krajnjeg čvora. Datoteka *upis.txt*, data u prilogu ovog rada, koja se koristi u svim kodovima sadrži opis grafa sa slike 3. Prve dvije linije predstavljaju broj čvorova i broj grana, nakon toga linije opisuju cijenu puteva od susjednih čvorova i kako su usmjereni, a zadnje dvije linije datoteke sadrže broj početnog i krajnjeg čvora (u ovom primjeru 0-ti i 11-ti čvor). U svim kodovima zamjenom `fin` sa `std::cin` i `fout` sa `std::cout` će se preći sa datotečnog upisa-ispisa na konzolni upis-ispis.

Datoteka se učitava kroz kod preko `std::ifstream` toka iz `<fstream>` biblioteke, a rezultat se upisuje u novu .txt datoteku preko `std::ofstream` toka iz istoimene biblioteke.

Glavni dio implementacije Bellman-Ford algoritma je dat u nastavku pod Kod 1 (svi kodovi unutar rada koriste temu `solarized light`[5]), prije čega se inicijaliziraju potrebne vrijednosti za broj čvorova i broj grana, kao što se i unesu vrijednosti cijena puteva unutar grafa (što se uradi preko tokova kako je opisano u prethodnom pasusu).

Kao što se može vidjeti, prvo se inicijaliziraju cijene svih grana (`std::vector<int> distance` u kodu) na ∞ tj. na 2^{29} zbog toga što ako se dvije takve putanje sabere, onda se ne prekorači maksimalna vrijednost `int` tipa u C++ programskom jeziku koja iznosi $2^{31} - 1$. Vrijednost 2^{29} je svakako veća od bilo koje putanje unutar grafa, što simbolički označava ∞ . Za početni čvor stavimo da je udaljen za cijenu 0 od samog sebe.

Ukoliko možemo prolaskom kroz čvor sa indeksom `ivice.at(j).first` (`std::vector<std::pair<int, int>>` `ivice` inicijaliziran prije glavnog dijela algoritma kroz datoteku *upis.txt*) i `ivice` sa indeksom `j` (na isti način inicijaliziran vektor `std::vector<int> tezine_ivica` smanjiti udaljenost čvora sa indeksom `ivice.at(j).second` od početnog čvora, to se i uradi.

Varijabla `flag` koja je `boolean` tipa služi kako bi algoritam završio sa izvršavanjem čim je izvjesno da su pronađene

najkraće putanje od početnog čvora.

Druga varijabla *boolean* tipa, varijabla u kodu *flag_neg_ciklusi*, služi za detekciju negativnih ciklusa - ukoliko nakon pronalaska najkraćih puteva prolaskom kroz sve ivice neka od udaljenosti čvorova od početnog čvora se ažurira, onda po Teoremu 2.2 graf sadrži negativan ciklus i ne postoje najkraće putanje od početnog čvora ka svim ostalim čvorovima u grafu.

Kod 1. Implementirani kod za Bellman-Ford algoritam

```
1 // Za svaki cvor se pamtni koliko je
  udaljen od pocetnog cvora
2 // Inicijalizirani su na 2^29 (beskonacno
  za nase svrhe)
3 std::vector<int> distance(br_cvorova, 1 <<
  29);
4 // Za pocetni cvor znamo da nije beskonacno
  udaljen od samog sebe:
5 distance.at(pocetni_cvor) = 0;
6
7 int broj_iteracija = 0;
8 for(int i = 0; i < br_cvorova - 1; i++) {
9     bool flag = false;
10    for(int j = 0; j < br_ivica; j++) {
11        if(distance.at(ivice.at(j).first) +
            tezine_ivica.at(j) <
            distance.at(ivice.at(j).second))
12        {
            distance.at(ivice.at(j).second)
                = distance.at(
                    ivice.at(j).first) +
                    tezine_ivica.at(j);
13            flag = true;
14        }
15        broj_iteracija++;
16    }
17    if(!flag) break;
18 }
19
20 bool flag_neg_ciklusi = false;
21 for(int j = 0; j < br_ivica; j++) {
22     if(distance.at(ivice.at(j).first) +
        tezine_ivica.at(j) <
        distance.at(ivice.at(j).second)) {
23         flag_neg_ciklusi = true;
24     }
25     broj_iteracija++;
26 }
```

U nastavku je prikazana izlazna .txt datoteka za izvršeni Bellman-Ford algoritam nad navedenim primjerom pod Kod 2.

Kod 2. Rezultat ispisa za Bellman-Fordov algoritma

```
1 *****
2 Najkraca distanca od 0-og do 11-og cvora
  iznosi: 8.
3 U grafu nema negativnih ciklusa.
4
5 Kompleksnost Bellman-Ford algoritma je O(broj
  cvorova * broj ivica) u najgorem slucaju.
6 Izracunate su najkrace udaljenosti od pocetnog
  do svakog drugog cvora.
7
8 Broj iteracija algoritma je bio 69 (jedna
  iteracija je poredjenje dvije vrijednosti
  duzina).
9 *****
```

Glavni dio implementacije Dijkstrinog algoritma je dat u nastavku pod Kod 3. Prije svega, za razliku od ostalih

algoritama koji su prikazani u ovom radu, implementacija Dijkstrinog algoritma uključuje napredniju strukturu podataka koja se zove *std::priority_queue*. *std::priority_queue* omogućuje brz pristup najvećem elementu koji se nalazi u njemu. Također podržava brzo ubacivanje elemenata i brzo izbacivanje najvećeg elementa (sve sa $O(\log n)$ kompleksnosti).

Kako nam je potreban pristup najmanjem elementu, onda je potrebno modificirati *std::priority_queue* tako da umjesto brzog prisupa najvećem elementu, imamo brz pristup najmanjem elementu. To se radi sljedećom konstrukcijom *std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int> >, std::greater<std::pair<int, int> > >*, u slučaju da su elementi *std::priority_queue*-a tipa *std::pair<int, int>*, što je i implementirano unutar koda. Pri poređenju dvije varijable tipa *std::pair<int, int>*, prvo se upoređuju prvi elementi parova, pa onda drugi.

Sa takvim tipom elementa onda se *std::priority_queue* inicijalizira sa početnim čvorom i njegovom udaljenosti od samog sebe koja iznosi 0. Ovo se vrši komandom *iduci_cvor.push(std::make_pair(0, pocetni_cvor))*. Primijetimo da je prvi element težina, a drugi element indeks čvora. Ovo je urađeno kako bi *std::priority_queue* držao čvorove u željenom poretku.

Sada se ponavlja idući postupak unutar koda: posmatrajući čvor koji je najbliži početnom iz *std::priority_queue*-a, svi njegovi susjedi koji nisu do sada posmatrani se dodaju u *std::priority_queue* sa težinom jednakom zbiru težine trenutnog posmatranog čvora i težine veze između njih. To se postiže linijom *iduci_cvor.push(std::make_pair(trenutni_cvor.first + x.second, x.first))*.

Prvim posmatranjem konačnog čvora poznata je dužina optimalnog puta do njega. Ova činjenica nije iskorištena za optimizaciju koda ispod. Umjesto toga, korišten je jednostavniji kriterij zaustavljanja - ispražnjenje *std::priority_queue*-a.

Kod 3. Implementirani kod za Dijkstrin algoritam

```
1 // Pravimo priority queue da bismo znali
  koji je preostali cvor najblizi pocetnom
2 // Ovako ide konstrukcija ako zelimo da
  priority queue daje prednost najmanjem,
  a ne najvećem
3 std::priority_queue<std::pair<int, int>,
  std::vector<std::pair<int, int> >,
  std::greater<std::pair<int, int> > >
  iduci_cvor;
4 std::vector<bool> posjecen(br_cvorova,
  false);
5
6 // Pocetni cvor je uvijek 0 udaljen od
  samog sebe:
7 iduci_cvor.push(std::make_pair(0,
  pocetni_cvor));
8 int udaljenost = -1;
9
10 int broj_iteracija = 0;
11 std::pair<int, int> trenutni_cvor;
12 while(!iduci_cvor.empty()) {
13     trenutni_cvor = iduci_cvor.top();
14     iduci_cvor.pop();
15     if(posjecen.at(trenutni_cvor.second) )
        continue;
```

```

16     posjecen.at(trenutni_cvor.second) =
17         true;
18     if(trenutni_cvor.second ==
19         krajnji_cvor) {
20         udaljenost = trenutni_cvor.first;
21     }
22     // Moramo proci kroz sve susjede od
23     trenutni_cvor.first
24     for(auto x :
25         susjedstva.at(trenutni_cvor.second))
26     {
27         if(posjecen.at(x.first)) continue;
28         iduci_cvor.push( std::make_pair(
29             trenutni_cvor.first + x.second,
30             x.first ) );
31         broj_iteracija++;
32     }
33 }

```

U nastavku je prikazana izlazna .txt datoteka za izvršeni Dijkstrin algoritam nad navedenim primjerom pod Kod 4.

Kod 4. Rezultat ispisa za Dijkstrin algoritam

```

1  *****
2  Najkraca distanca od 0-og do 11-og cvora
3  iznosi: 8.
4  Kompleksnost Dijkstra algoritma je O(broj ivica
5  + broj cvorova * log(broj cvorova)) u
6  najgorem slucaju.
7  Izracunata je samo potrebna distanca.
8  Broj iteracija algoritma je bio 14 (jedna
9  iteracija je poredjenje dvije vrijednosti
10  duzina).
11 *****

```

Glavni dio Floyd-Warshallovog algoritma je dat pod Kod 5. Ovaj algoritam koristi tzv. matricu susjedstva kako bi se držale procjene udaljenosti između svaka dva čvora u grafu. Ovaj algoritam radi na sljedeći način: posmatraju se sve trojke čvorova grafa (A , B , C). Ukoliko je zbir udaljenosti od A do B i od B do C manji od trenutne procijenjene udaljenosti od A do C , onda se ažurira udaljenost od A do C .

Na početku su sve međusobne udaljenosti postavljene na ∞ , osim udaljenosti čvorova od samih sebe i od čvorova sa kojima imaju neposrednu vezu putem jedne od grana grafa.

Kod 5. Implementirani kod za Floyd-Warshallov algoritam

```

1  int broj_iteracija = 0;
2  for(int i = 0; i < br_cvorova; i++) {
3      for(int j = 0; j < br_cvorova; j++) {
4          for(int k = 0; k < br_cvorova; k++) {
5              if(matrica_udaljenosti.at(i
6                  ).at(j) +
7                  matrica_udaljenosti.at(j
8                  ).at(k) <
9                  matrica_udaljenosti.at(i
10                     ).at(k))
11                  matrica_udaljenosti.at(i
12                      ).at(k) =
13                      matrica_udaljenosti.at(i
14                          ).at(j) +
15                      matrica_udaljenosti.at(j
16                          ).at(k);
17                  broj_iteracija++;
18              }
19          }
20      }
21  }

```

U nastavku je prikazana izlazna .txt datoteka za izvršeni Floyd-Warshallov algoritam nad navedenim primjerom pod Kod 6.

Kod 6. Rezultat ispisa za Dijkstrin algoritam

```

1  *****
2  Najkraca distanca od 0-og do 11-og cvora
3  iznosi: 8.
4  Kompleksnost Floyd-Marshall algoritma je
5  O((broj cvorova)^3) u najgorem slucaju.
6  Izracunate su najkrace udaljenosti od svakog
7  pocetnog do svakog drugog cvora.
8  Broj iteracija algoritma je bio 1728 (jedna
9  iteracija je poredjenje dvije vrijednosti
10  duzina).
11 *****

```

Kao što se može vidjeti, Dijkstrin algoritam je u najmanje iteracija došao do rezultata za 14 iteracija, Bellman-Fordov algoritam je za 69 iteracija došao do rezultata, dok je Floyd-Warshallov algoritam došao do rezultata za 1728 iteracija.

Dobiveni rezultat je očekivan zbog toga što Dijkstrin algoritam računa samo potrebnu distancu (tj. samo distancu od 0-tog do 11-tog čvora), Bellman-Ford algoritam računa najkraće udaljenosti od početnog, 0-tog čvora u ovom slučaju, do svakog drugog čvora unutar grafa, dok Floyd-Warshallov algoritam je izračunao sve najkraće udaljenosti od bilo kojeg čvora do drugog bilo kojeg čvora u grafu.

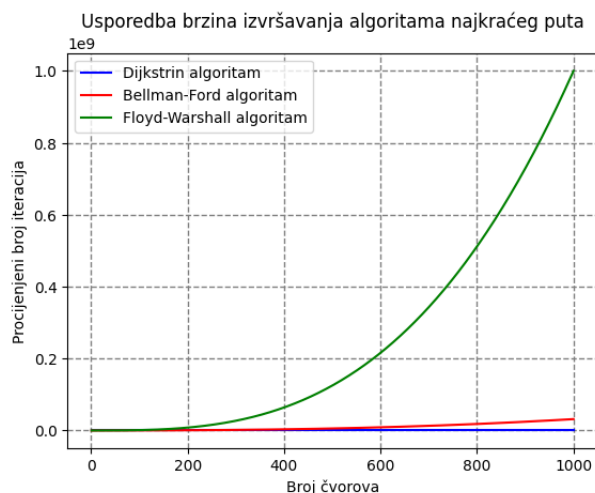
Na slikama 4, 5, 6 i 7 je prikazana usporedba procijenjenog broja iteracija svakog od ova tri algoritma. Procjene su napravljene direktno preko formula iz kompleksnosti ovih algoritama.

Ukoliko je broj čvorova n , onda je uzeta aproksimacija da je broj grana jednak $n \cdot \sqrt{n}$ (ovo je uzeto kao procjena grafa koji nije ni naročito gust ni naročito rijedak).

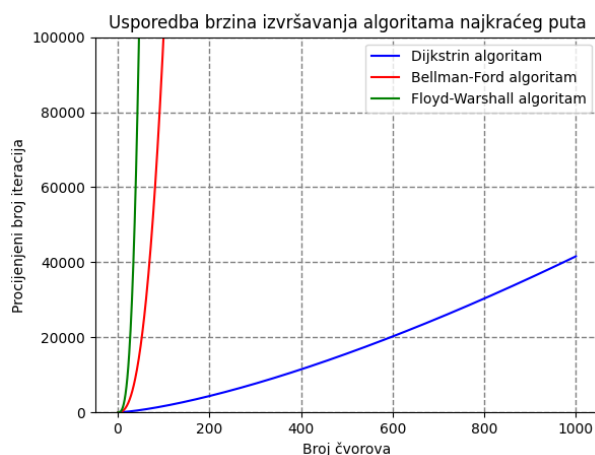
Sa grafika se jasno može vidjeti da je Dijkstrin algoritam najbrži, dok je Floyd-Warshall algoritam najsporiji. Bellman-Ford algoritam nudi dobar kompromis između brzine izvršavanja i opsega primjenjivosti. Također, on je jedini od ova tri algoritma koji je pogodan za primjenu nad distribuiranim sistemima.

ZAKLJUČAK

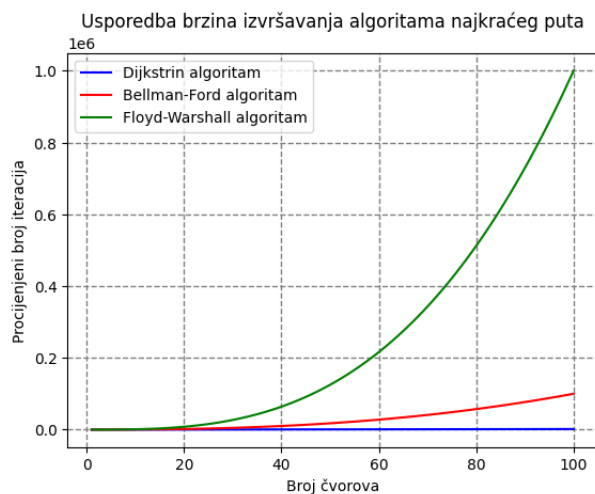
U radu je opisan problem pronalaska najkraćeg puta u usmjerenim težinskim grafovima. Za rješenje ovog problema predloženi su Dijkstrin algoritam kao i Floyd-Warshallov algoritam. Međutim uočeno je da ovi algoritmi imaju određena ograničenja kada su u pitanju grafovi sa negativnim težinama ili kada grafovi sadrže negativne cikluse. Iz tog razloga ovaj rad se fokusirao na Bellman-Fordov algoritam koji se može primijeniti upravo na takve grafove. Najprije je dat matematski opis i dokaz algoritma kao i pseudokod algoritma, a zatim implementacija u proramskom jeziku C++. U svrhu poređenja Bellman-Fordovog algoritma također su razvijene implementacije Dijkstrinog algoritma kao i Floyd-Warshallov algoritma. C++ je korišten sa ciljem pouzdanijeg vremena izvršavanja (pošto je kompajlirani jezik i nema *garbage collector*-a). No, kako su grafovi koji su korišteni mali, vrijeme izvršavanja se nije moglo precizno odrediti, tako da smo uporedili broj



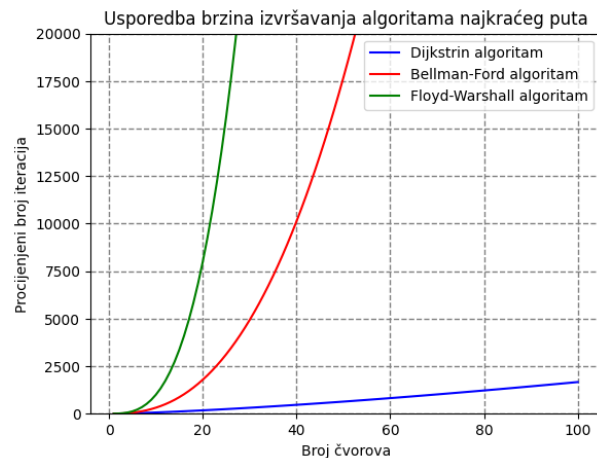
Slika 4. Usporedba brzina izvršavanja Dijkstrinog, Bellman-Ford i Floyd-Warshall algoritma za broj čvorova do 1000



Slika 5. Usporedba brzina izvršavanja Dijkstrinog, Bellman-Ford i Floyd-Warshall algoritma za broj čvorova do 1000, ali je grafik uvećan



Slika 6. Usporedba brzina izvršavanja Dijkstrinog, Bellman-Ford i Floyd-Warshall algoritma za broj čvorova do 100



Slika 7. Usporedba brzina izvršavanja Dijkstrinog, Bellman-Ford i Floyd-Warshall algoritma za broj čvorova do 100, ali je grafik uvećan

iteracija pri izvršavanju svakog od algoritama, te su prokommentarisani očekivani rezultati.

Kako bi se maksimizirala iskoristivost svakog od ovih algoritama, potrebno ih je koristiti u odgovarajućim kontekstima, dakle ako je potrebno da se nađe samo najkraća udaljenost od nekog početnog čvora do nekog drugog krajnjeg čvora u grafu bez negativnih grana, onda bi tu Dijkstrin algoritam bio najpogodniji, dok bi Floyd-Warshallov algoritam tu bio suvišan.

Bitno je naglasiti da iako je Dijkstrin algoritam najbrži, ipak Bellman-Ford algoritam može detektovati negativne cikluse, dok Dijkstrin algoritam ne može raditi u grafovima sa negativnim težinama granama. Floyd-Warshall algoritam može detektovati i negativne cikluse, ali nije pogodan za to jer se onda mora modificirati.

LITERATURA

- [1] Željko Jurić. *Predavanja na predmetu Diskretna matematika*. Elektrotehnički fakultet, Univerzitet u Sarajevu, 2022.
- [2] J. Bang-Jensen i G.Z. Gutin. *Digraphs: Theory, Algorithms and Applications*. Springer Monographs in Mathematics. Springer London, 2008. ISBN: 9781848009981.
- [3] prof. dr željko Jurić. *Pomoćna prezentacija uz predavanja - Bellman-Fordov algoritam*. <https://c2.etf.unsa.ba/course/view.php?id=52>. Pristup stranici: 02. juli 2023. 2022.
- [4] prof. Steven Halim. *visualgo.net*. <https://visualgo.net/en/>. Pristup stranici: 02. juli 2023. 2011.
- [5] Ethan Schoonover. *Solarized*. <http://ethanschoonover.com/solarized>. Pristup stranici: 20. maj 2023. 2011.